CSS Course

Comprehensive Notes

Emmanuel Adedayo

Code Academy | SEA Technologies Media

Learn. Build. Innovate.

SEA TECHNOLOGIES MEDIA

Topic 1: Introduction to CSS and the 3 Ways to Add It

CSS stands for Cascading Style Sheets. It is the language that controls how HTML elements look on the screen — their colours, sizes, spacing, fonts, and layout. If HTML is the skeleton of a web page, CSS is the skin, clothing, and styling that makes it presentable.

Key Concepts

- **CSS = Cascading Style Sheets** — *Cascading* means styles flow down and later or more specific rules can override earlier ones.
- **A CSS rule** has two parts: a **selector** (what to style) and a **declaration block** (how to style it), written as `selector { property: value; }`.
- **Property** — the aspect you want to change, e.g. `color`, `font-size`, `margin`.
- **Value** — what you set that property to, e.g. `blue`, `18px`, `20px`.
- **Every declaration ends with a semicolon** — a missing semicolon breaks the declarations that follow it.
- **Comments** use `/* ... */` — CSS has no single-line `//` comment.

There are three ways to add CSS to a web page:

- **Inline CSS** — written inside an element's `style` attribute, e.g. `<p style="color: red;">`. It affects only that one element. Useful for a quick test, but it mixes style with structure and cannot be reused.
- **Internal CSS** — written inside a `<style>` tag in the `<head>` of the HTML file. It styles the whole page and is handy for a single-page project.
- **External CSS** — written in a separate `.css` file and linked from the HTML with a `<link>` tag that carries `rel="stylesheet"` and `href="style.css"`. This is the professional standard: one stylesheet can style an entire website, and the browser caches it for faster loading.

When two rules target the same element, the browser resolves the conflict using the cascade: inline styles win over internal and external styles, and when specificity is equal, the rule written last wins.

Tip

Always Prefer External CSS
Keeping your styles in a separate `style.css` file means you edit one file to restyle every page. It also keeps your HTML clean and readable — structure in HTML, presentation in CSS.

Warning

Mind the Semicolon and the Link Tag
Two mistakes beginners make constantly: forgetting the semicolon at the end of a declaration, and misspelling the `<link>` tag or its `href` path. If your CSS "is not working", check the link path first — the file name and folder must match exactly, including capital letters.

Key takeaway: CSS controls the presentation of HTML. A rule is a selector plus property–value declarations. You can add CSS inline, internally, or externally — and external

> stylesheets are the best practice for real projects.

Topic 2: Colors and Backgrounds

Colour is one of the fastest ways to change the feel of a page. CSS gives you several ways to describe a colour, and several properties for painting backgrounds.

Key Concepts

- **Named colours** — about 140 keywords such as `red`, `tomato`, `steelblue`. Easy to read, limited in range.
- **HEX** — `#RRGGBB`, e.g. `#1E56D6`. Each pair is a red, green, or blue value from `00` to `FF`. Shorthand `#FFF` equals `#FFFFFF`.
- **RGB** — `rgb(30, 86, 214)`, three channels from 0 to 255.
- **RGBA** — the same plus an alpha channel for transparency, from `0` (invisible) to `1` (solid), e.g. `rgba(0, 0, 0, 0.4)`.
- **HSL** — `hsl(220, 75%, 48%)`: hue in degrees, saturation and lightness in percent. Excellent for building colour variations.
- **`color`** sets text colour; **`background-color`** paints the element's box.
- **`opacity`** — `0` to `1`, and it fades the whole element including its children.
- **Gradients** — `linear-gradient()` and `radial-gradient()` are images, so they belong to `background` or `background-image`, not `background-color`.

Background images are controlled by a family of properties. `background-image` sets the source and `background-size` scales it, with `cover` and `contain` being the two values you will reach for most. `background-position` places the image, and `background-repeat` decides whether it tiles. The `background` shorthand can combine all of them in a single line.

Tip

RGBA vs Opacity
Use `rgba()` when you want a see-through *background* but crisp, fully opaque text on top. Use `opacity` only when you genuinely want to fade the entire element, text and all.

Warning

Contrast Is Accessibility
Light grey text on a white background may look elegant on your screen and be unreadable for many users. Aim for a contrast ratio of at least 4.5:1 for body text. Never rely on colour alone to convey meaning — add an icon, label, or underline as well.

Key takeaway: Describe colours with names, HEX, RGB/RGBA, or HSL; paint text with `color` and boxes with `background-color`. Reach for `rgba()` for transparent backgrounds, and always check your contrast.

Topic 3: Typography

Typography is how your text looks and reads. Good typography makes content effortless to scan; poor typography makes even great writing feel like work.

Key Concepts

- **`font-family`** — takes a comma-separated *font stack*, e.g. `'Inter', Arial, sans-serif`. The browser uses the first font it has, so always end with a generic family (`sans-serif`, `serif`, `monospace`).
- **Quote multi-word font names** — `'Times New Roman'`, not `Times New Roman`.
- **`font-size`** — `px` is fixed; `rem` scales with the root font size and is the modern default for accessible text.
- **`font-weight`** — keywords `normal` / `bold`, or numbers `100`–`900`.
- **`font-style`** — `normal`, `italic`, or `oblique`.
- **`line-height`** — vertical space between lines. A unitless value such as `1.6` is best because it scales with the font size.
- **`text-align`** — `left`, `right`, `center`, `justify`.
- **`text-transform`** — `uppercase`, `lowercase`, `capitalize`.
- **`letter-spacing`** and **`word-spacing`** — fine-tune the space between characters and words.
- **`text-decoration`** — most often `none` to remove the default underline from links.

Web fonts from services such as Google Fonts are added with a `<link>` in your HTML `<head>` or with `@import` at the very top of your stylesheet, then referenced in `font-family` like any other font.

Tip

A Readable Default
Body text around `1rem` (16px) with `line-height: 1.6` and a line length of 60–75 characters is comfortable for almost everyone. Set the type before you fuss over colours — it affects readability far more.

Warning

Do Not Remove Link Underlines Carelessly
`text-decoration: none` is fine for navigation menus, but inside body text the underline is how users recognise a link. If you remove it, give the link another strong visual cue such as a distinct colour plus a hover style.

Key takeaway: Control text with `font-family`, `font-size`, `font-weight`, and `line-height`. Always provide a font stack ending in a generic family, and prefer `rem` with a unitless line height.

Topic 4: The Box Model

Every element on a web page is a rectangular box. The box model describes the four layers of that box and is the single most important concept for understanding CSS layout and spacing.

Key Concepts

- **Content** — the text or image itself, sized by `width` and `height`.
- **Padding** — space *inside* the box, between the content and the border. It takes the element's background colour.
- **Border** — the line around the padding. Written as `border: 2px solid #1E56D6` (width, style, colour) — the **style is required**.
- **Margin** — space *outside* the border that pushes other elements away. It is always transparent.
- **Shorthand order is clockwise** — `padding: top right bottom left`. Two values mean vertical then horizontal; three mean top, horizontal, bottom.
- **`box-sizing: border-box`** — makes `width` include padding and border, so a box you declare as `300px` actually occupies `300px`.
- **`margin: 0 auto`** — centres a block element that has a set width.
- **`border-radius`** rounds corners; `50%` on a square makes a circle.

By default (`box-sizing: content-box`), the visible width of an element is `width + padding-left + padding-right + border-left + border-right`. That arithmetic is the cause of countless "why is my layout 40px too wide?" bugs, which is why most projects start with a global reset.

Tip

The Universal Reset
Begin every stylesheet with `* { margin: 0; padding: 0; box-sizing: border-box; }`. It removes inconsistent browser defaults and makes every width behave the way you expect.

Warning

Margin Collapse
Vertical margins between two block elements collapse into one: a `30px` bottom margin next to a `20px` top margin produces `30px` of space, not `50px`. When you need guaranteed separation, use padding on a wrapper or set the margin on only one side consistently.

Key takeaway: Content, padding, border, margin — inside to outside. Padding is inner space, margin is outer space, and `box-sizing: border-box` keeps your declared widths honest.

Topic 5: Selectors and Specificity

Selectors decide *which* elements a rule applies to. Specificity decides *which rule wins* when several rules target the same element. Master both and CSS stops feeling unpredictable.

Key Concepts

- **Element selector** — `p`, `h1`, `ul`: styles every element of that type.
- **Class selector** — `.card`, written with a dot. Reusable on many elements; your everyday tool.
- **ID selector** — `#header`, written with a hash. Must be unique on the page and is very hard to override.
- **Attribute selector** — `input[type="text"]` targets elements by attribute value.
- **Grouping** — `h1, h2, h3 { ... }` applies one rule to several selectors.
- **Descendant** — `nav a` (a space) matches any `a` inside a `nav`, at any depth.
- **Child** — `nav > a` matches only direct children.
- **Pseudo-classes** — `:hover`, `:focus`, `:first-child`, `:nth-child(2n)` style elements in a particular state or position.
- **Pseudo-elements** — `::before` and `::after` (two colons) insert generated content.

Specificity is scored roughly as inline style (1000) > ID (100) > class, attribute, or pseudo-class (10) > element (1). When two rules score equally, the one written later in the stylesheet wins — that is the ”cascade” in Cascading Style Sheets.

Tip

Build with Classes
Classes hit the sweet spot: specific enough to be precise, low enough in specificity to override easily. If you find yourself fighting your own CSS, you are probably using IDs where a class would do.

Warning

Avoid `!important`
`!important` overrides everything and makes future bugs much harder to fix, because the only way to beat it is another `!important`. Fix the specificity or the rule order instead — reserve `!important` for genuine emergencies with third-party code.

Key takeaway: Use element, class, ID, attribute, and pseudo-class selectors to target elements precisely. Remember the specificity order inline > ID > class > element, and lean on classes to keep your styles maintainable.

Topic 6: Display and Visibility

The `display` property is the master switch of CSS layout: it decides how an element behaves in the flow of the page and how it treats its own children.

Key Concepts

- **`block`** — starts on a new line and fills the available width. Accepts `width`, `height`, and all margins. Examples: `div`, `p`, `h1`.
- **`inline`** — flows within a line of text, only as wide as its content. **Ignores `width` and `height`** and vertical margins. Examples: `span`, `a`, `strong`.
- **`inline-block`** — flows inline like text but accepts `width`, `height`, and padding. Perfect for buttons and nav items.
- **`none`** — removes the element from the page completely; it occupies no space at all.
- **`flex`** and `grid` — turn the element into a layout container for its children.
- **`visibility: hidden`** — hides the element but **keeps its space** reserved.
- **`overflow`** — `visible`, `hidden`, `scroll`, or `auto` controls content that is too big for its box.

The difference between the two ways of hiding things matters in practice. Use `display: none` for content that should not exist right now, such as a closed dropdown menu. Use `visibility: hidden` when the layout must not shift — for example an element that appears and disappears inside a fixed grid.

Tip

Reach for `inline-block` for Nav Links
An `<a>` is inline, so vertical padding on it looks cramped and overlaps neighbouring lines. Set `display: inline-block` (or make the parent a flex container) and your padding, width, and height start working properly.

Warning

Hidden Is Not Secure
`display: none` only hides content visually — it is still in the HTML and anyone can read it in the page source. Never use it to hide passwords, prices, or any sensitive data. Also note screen readers skip `display: none` content, which can be either what you want or an accessibility bug.

Key takeaway: `display` decides how an element sits in the flow: `block` stacks, `inline` flows, `inline-block` does both. `display: none` removes the space, `visibility: hidden` keeps it.

Topic 7: Flexbox

Flexbox is a one-dimensional layout system that arranges items in a row or a column and distributes the space between them. It replaced years of float and table hacks and is the tool you will reach for daily.

Key Concepts

- **`display: flex`** — turns an element into a flex container; its direct children become flex items.
- **`flex-direction`** — `row` (default), `column`, `row-reverse`, `column-reverse`. It defines the *main axis*.
- **`justify-content`** — aligns items **along the main axis**: `flex-start`, `center`, `flex-end`, `space-between`, `space-around`, `space-evenly`.
- **`align-items`** — aligns items **along the cross axis**: `stretch` (default), `center`, `flex-start`, `flex-end`, `baseline`.
- **`flex-wrap`** — `nowrap` (default) or `wrap` to let items move onto a new line.
- **`gap`** — clean, modern spacing between items with no margin arithmetic.
- **`flex-grow` / `flex-shrink` / `flex-basis`** — how an item grows, shrinks, and starts out; the `flex: 1` shorthand makes items share space equally.
- **`align-self`** — overrides `align-items` for one individual item.

Perfect centring, once the hardest trick in CSS, is now three lines on the parent: `display: flex; justify-content: center; align-items: center;`. Remember that `justify-content` and `align-items` swap their visual direction when you change `flex-direction` to `column`.

Tip

Think in Axes, Not in Left and Right
Do not memorise "justify is horizontal". Memorise "justify follows the main axis, align follows the cross axis". Then flexbox behaves consistently whether you are in a row or a column.

Warning

Flex Only Affects Direct Children
`display: flex` applies to the container's immediate children only — grandchildren are unaffected. If a nested element is not aligning, that element's own parent probably needs to be a flex container too.

Key takeaway: Set `display: flex` on the parent and choose a direction, then use `justify-content` for the main axis, `align-items` for the cross axis, and `gap` for the spacing. It is the fastest way to build clean, responsive rows and columns.

Topic 8: Positioning

Positioning takes an element out of, or shifts it within, the normal document flow. It is how you build sticky headers, dropdown menus, badges, modals, and tooltips.

Key Concepts

- **`static`** — the default. The element sits in normal flow and ignores `top`, `right`, `bottom`, `left`.
- **`relative`** — stays in the flow but can be nudged with the offset properties. Its original space is preserved. Crucially, it becomes the **reference point** for absolutely positioned children.
- **`absolute`** — removed from the flow and positioned against its nearest *positioned* ancestor (or the page if there is none).
- **`fixed`** — removed from the flow and pinned to the viewport; it stays put when the page scrolls. Ideal for navbars and floating buttons.
- **`sticky`** — behaves like `relative` until it hits a scroll threshold, then acts like `fixed`. Requires a `top` (or other offset) value to work.
- **Offsets** — `top`, `right`, `bottom`, `left` do nothing unless `position` is something other than `static`.
- **`z-index`** — stacking order for overlapping positioned elements; higher numbers sit on top.

The classic pattern is a parent with `position: relative` and a child with `position: absolute`. That is how you place a "New" badge on the corner of a card, or a close button inside a modal: the parent defines the coordinate system, the child places itself inside it.

Tip

Fixed Headers Need Body Padding
A `position: fixed` header is lifted out of the flow, so the content beneath it slides up and hides behind it. Add `padding-top` to the body (or a top margin to the first section) equal to the header's height.

Warning

`z-index` Needs Positioning
`z-index` is ignored on a `static` element. If your overlay refuses to come to the front, check that it has `position: relative`, `absolute`, or `fixed`. Also avoid huge values like `99999` — keep a simple, documented scale such as 10, 100, 1000.

Key takeaway: `static` is default, `relative` nudges and anchors, `absolute` positions against a positioned ancestor, `fixed` pins to the viewport, and `sticky` switches between the two. Offsets and `z-index` only work on positioned elements.

Topic 9: Responsive Design and CSS Variables

Responsive design means one stylesheet that looks right on a phone, a tablet, and a desktop monitor. CSS variables (custom properties) let you define your design decisions once and reuse them everywhere.

Key Concepts

- **Declare a variable** inside `:root` so the whole document can see it: `--primary: #1E56D6;`. The two leading dashes are required.
- **Use a variable** with the `var()` function — `color: var(--primary);`. The dashes are required inside `var()` too.
- **Fallback value** — `var(--primary, #1E56D6)` uses the second value if the variable is not defined.
- **Media query** — `@media (max-width: 768px) { ... }` applies rules only at certain screen sizes.
- **Common breakpoints** — roughly 480px (phone), 768px (tablet), 1024px (small laptop), 1200px (desktop).
- **Relative units** — `%` of the parent, `rem` of the root font size, `em` of the element's own font size, `vw` / `vh` of the viewport.
- **The viewport meta tag** — `<meta name="viewport" content="width=device-width, initial-scale=1.0">` in your HTML is mandatory, or mobile browsers will zoom out and your media queries will never fire.
- **`max-width`** — prefer `max-width: 1200px` with `margin: 0 auto` over a fixed `width` so containers can shrink on small screens.

Two strategies exist. *Mobile-first* writes the small-screen styles as the base and uses `min-width` queries to add complexity for larger screens — this is the modern recommendation. *Desktop-first* starts wide and uses `max-width` queries to simplify downwards.

Tip

Variables Make Theming Trivial
Put your colours, spacing scale, and border radius in `:root`. Changing your brand colour then means editing one line instead of hunting through hundreds. You can also redefine the same variables inside a `.dark-theme` class to get a dark mode almost for free.

Warning

Two Silent Breakers
First, `var(primary)` does nothing — the dashes are part of the name, so it must be `var(--primary)`. Second, without the viewport meta tag your site will look like a shrunken desktop page on a phone no matter how good your media queries are. Always test by narrowing your browser window.

Key takeaway: Define your design tokens in `:root` and consume them with `var(--name)`. Use relative units, `max-width` containers, media queries, and the viewport meta tag to make one layout work on every screen.

★ Topic 10: Final Project — Developer Portfolio

This final project pulls together every technique from the course into one polished, responsive page you can genuinely publish and show to employers.

Project Overview:

The Developer Portfolio is a single-page site with a fixed navigation bar, a full-height hero section, an about section, a responsive project card grid, a contact area, and a footer. Colours and spacing come from CSS variables, layout comes from Flexbox, and media queries make it work from a 360px phone up to a wide desktop.

Key Concepts

- **Variables** — brand colours, spacing, and `--radius` declared once in `:root` and reused throughout.
- **Box model** — a universal reset with `box-sizing: border-box`, then padding and margin for consistent rhythm.
- **Typography** — a web font stack, `rem` sizes, and `line-height: 1.6` for readable body copy.
- **Colours and backgrounds** — a gradient hero, `rgba()` overlays, and solid card backgrounds.
- **Selectors** — classes for components, `:hover` for interactive feedback, `::after` for decorative underlines.
- **Flexbox** — the navbar, the hero content, and the card grid rows, with `gap` for spacing.
- **Positioning** — `position: fixed` navbar with a `z-index`, and `position: relative` cards holding absolutely positioned badges.
- **Responsive** — media queries at 768px and 480px collapse the grid to one column and stack the navigation.

Tip

Style Section by Section
Write the full HTML structure first, then style top to bottom: reset and variables, then navbar, hero, about, projects, contact, footer. Check the page in the browser after each section so you always know which change caused which result.

Warning

Test Narrow Early
Do not leave responsiveness until the end. Drag your browser window narrow every few minutes, or keep the device toolbar open in DevTools. Fixing a layout that was only ever tested at 1440px is far more work than building it responsively from the start.

Project Features:

1. Fixed navigation bar with hover effects and smooth scrolling links
2. Full-height hero section with a gradient background and call-to-action button

3. About section with a two-column Flexbox layout that stacks on mobile
4. Responsive project card grid with hover lift and shadow
5. Skills list styled with reusable tag components
6. Contact section with styled form inputs and focus states
7. Footer with social links
8. Media queries at 768px and 480px for tablet and phone

> **Key takeaway:** Your portfolio is the proof of everything you have learned: variables, the box model, typography, selectors, Flexbox, positioning, and responsive design working together. Build it carefully, publish it, and keep improving it as you grow.